AI First Nations
YOLO Mangrove Tree Counter

High Distinction Model Evaluation Specification & Assessment Rubric
Developer handover | Jupyter Notebook deliverable | 11 October 2026

Primary objective: Produce reproducible, quantitative evidence of how accurately the YOLO model detects and counts individual mangrove crowns, generalises to unseen drone imagery, and performs through the real Flask application pipeline.

Important: High Distinction describes the quality of the evaluation, not a guaranteed model accuracy. Never manipulate evaluation data or tune on the final test set to reach a target.

1. Assessment purpose and research questions

• How accurately does YOLO locate individual mangrove tree crowns?

• How closely do predicted tree counts match manually verified ground-truth counts?

• Does performance generalise to unseen source images, sites and surveys?

• How robust is the system to dense canopy, small crowns, shadows, resolution and tile boundaries?

• Can the evaluated weights run correctly and efficiently in the actual Flask upload/tiling/inference pipeline?

2. Required deliverables

Deliverable	Evidence required
Evaluation notebook	YOLO_Tree_Counter_Evaluation.ipynb; runs top-to-bottom
Ground truth and provenance	Annotated evaluation images, annotation quality checks, source/site identifiers and split manifests
Metrics and plots	Detection and counting metrics, PR curves, scatter plots, failure examples
Model experiments	Baseline vs improved model/setting comparison and locked test evaluation
Flask validation	Real end-to-end inference tests, overlay and count consistency, latency and memory
Exports and handover	Prediction CSVs, evaluation summaries, saved configuration, dependency instructions

3. Mandatory notebook sections

1. Introduction

Problem statement, existing pipeline, scope, five research questions, measurable evaluation criteria.

2. Dataset audit and EDA

Source images, sites, original sizes, crown counts, class/condition distribution, annotation examples and limitations.

3. Split integrity

Source-level train/validation/test split BEFORE tiling; detect duplicates, correlated images and site confounding.

4. Model setup

YOLO family/version/size, checkpoint, image size, augmentation, epochs, batch size, hardware, seed, dependencies and thresholds.

5. Detection evaluation

Precision, recall, F1, AP/mAP@0.5, mAP@0.5:0.95, PR curves, threshold sensitivity and failure examples.

6. Counting evaluation

Ground-truth vs predicted counts per tile, source image and survey; MAE, RMSE, bias and relative count errors.

7. Tiling and deduplication

640x640 tiling, edge coverage, original-image coordinates, boundary crowns, overlap effects and duplicate suppression.

8. Robustness and error analysis

Stratify where data permits: density, crown size, site, lighting, water, non-mangrove vegetation and resolution.

9. Improvement experiments

At least two controlled comparisons where feasible; tune only on training/validation data.

10. Application integration

Load same weights and settings as Flask; run upload-to-result workflow, inspect overlays, measure runtime/memory.

11. Conclusion

Final metrics table, limitations, threshold selection, evidence-backed Ready/Conditional/Not Ready decision.

4. High Distinction marking rubric (100 marks)

Criterion	Marks	HD standard
Research design	5	Clear objectives, measurable questions, defensible methodology
Dataset and EDA	10	Complete audit, annotation verification and meaningful visualisations
Leakage prevention	10	Source/site-aware splits, overlap and duplicate checks, reproducible proof
Model reproducibility	10	Complete weights, settings, environment and deterministic controls
Detection evaluation	15	Correct mAP, precision/recall/F1, threshold study and interpretation
Counting evaluation	15	Ground-truth MAE/RMSE/bias, image-level errors and visual evidence
Tiling and robustness	10	Boundary/coverage/dedup tests and hard-case analysis
Improvement experiments	10	Controlled baseline comparisons and critical interpretation
Flask validation	10	End-to-end functionality, output consistency, latency, memory, failures
Presentation/conclusion	5	Clear executable notebook, well-labelled charts and justified readiness

Grade	Marks	Standard
High Distinction	80-100	Rigorous, reproducible evaluation; strong evidence and critical analysis
Distinction	70-79	Comprehensive evaluation with minor omissions
Credit	60-69	Appropriate metrics but limited experiments or interpretation
Pass	50-59	Basic metrics with incomplete counting/integration validation
Fail	Below 50	Invalid evaluation, missing ground truth or unsupported claims

5. Mandatory evaluation metrics

Metric	Meaning / reporting
Precision = TP/(TP+FP)	Proportion of predicted crowns that match ground truth
Recall = TP/(TP+FN)	Proportion of ground-truth crowns detected
F1 = 2PR/(P+R)	Balance of precision and recall
mAP@0.5	Mean average precision at IoU 0.5 (single tree-crown class)
mAP@0.5:0.95	Mean AP across IoU 0.50 to 0.95 in 0.05 increments
MAE = mean(abs(pred - actual))	Average absolute count error per evaluated image
RMSE = sqrt(mean((pred - actual)^2))	Emphasises large count mistakes
Bias = mean(pred - actual)	Systematic overcounting (+) or undercounting (-)
Aggregate relative absolute count error	Sum absolute per-image errors divided by sum true counts; specify denominator and zero-count handling
Latency and peak memory	Measured on specified hardware and input size
Duplicate count	Counts before/after cross-tile deduplication, where applicable

Critical distinction: mAP evaluates spatial detection quality, not counting accuracy. A model can have the correct total count while missing real crowns and hallucinating others. Report both tasks separately. Include zero-tree negative examples and do not divide by zero for percentage errors.

6. Proposed performance targets (not guaranteed)

Metric	Aspirational target	Interpretation
mAP@0.5	At least 0.85	Detection target on held-out sources
Precision	At least 90%	At the validation-selected operating threshold
Recall	At least 85%	At the same threshold
Aggregate relative absolute count error	At most 10%	Non-empty image set; define aggregation precisely
mAP@0.5:0.95, MAE, RMSE, bias	Report measured values	No arbitrary universal threshold
Latency and memory	Agree hardware-specific limit	Measure real end-to-end pipeline

These are proposed engineering goals to discuss with the supervisor, not official academic cut-offs. HD-level evaluation is possible even if the model falls short, provided findings are rigorous and limitations are disclosed. Do not claim operational readiness solely from hitting numerical targets.

7. Mandatory experiments

A. Baseline

Evaluate current saved YOLO weights without tuning; capture full detection and counting metrics.

B. Confidence threshold

Compare thresholds (e.g., 0.10-0.90); plot precision, recall, F1 and counting error; select threshold on validation set and freeze before final test.

C. Tiling

Verify 640x640 output, full coverage, small/exact/large image cases, edge tiles, coordinate restoration and boundary-split crowns; compare overlap if justified.

D. Counting

Compare predicted and manually verified tree totals at source-image/survey level; scatter plot, error distribution and worst-case table.

E. Robustness

Assess dense/sparse canopy, crown sizes, shadows, water, different sites and resolution where sufficient labelled examples exist.

F. Improvement

At least two controlled comparisons where feasible: resolution, augmentation, annotation quality, YOLO variants or thresholds; no test-set optimisation.

G. Unseen imagery

Test genuinely unseen survey images. Unlabelled images can demonstrate inference but cannot establish quantitative accuracy.

8. Mandatory notebook figures

• Dataset and crown-count distributions; ground-truth overlay examples.

• Training/validation curves where history is available; precision-recall curve.

• Threshold vs precision/recall/F1 and counting MAE.

• Predicted vs true counts; error distribution; worst-performing images.

• False positives, false negatives, small/overlapping crown failures.

• Full-image before/after deduplication overlays and baseline/improved comparisons.

• Real Flask application output screenshot plus automated integration-test summary.

Every major figure must include an interpretation: what it shows, why it matters and which limitation remains.

9. Flask application integration acceptance tests

Required pipeline: Upload drone image → validate → tile at 640x640 → run saved YOLO → convert detections to original-image coordinates → deduplicate where necessary → calculate estimated count → render annotated overlay → display in Flask.

• Load the actual saved weights without unexpected downloads or retraining.

- Use identical preprocessing, model settings, threshold and deduplication configuration in notebook and Flask.
- Confirm numerical count/detection consistency for the same test image within documented tolerances.
- Validate full-image coordinate reconstruction and image-overlay alignment.
- Record end-to-end latency and peak memory on the intended hardware.
- Test unsupported, corrupt, oversized images and unavailable model weights; report graceful failure.
- Create a Pass/Fail table of expected vs observed outcomes. Screenshots alone are insufficient.

10. Required final results table

Metric	Baseline	Final	Target / decision
Precision	Measured	Measured	At least 90%
Recall	Measured	Measured	At least 85%
F1	Measured	Measured	Report
mAP@0.5	Measured	Measured	At least 0.85
mAP@0.5:0.95	Measured	Measured	Report
Counting MAE / RMSE / bias	Measured	Measured	Report all
Relative absolute count error	Measured	Measured	At most 10%
Inference time / memory	Measured	Measured	Agreed hardware target
Flask integration	Tested	Tested	All critical checks pass

Do not insert invented performance values. The conclusion must answer: detection accuracy; counting accuracy; generalisation; limitations; application performance; and a justified Ready, Conditional/Experimental or Not Ready recommendation.

11. Non-negotiable HD requirements

- No train/test leakage from tiles, source images or near-duplicate surveys.
- No counting-accuracy claim without verified tree-count ground truth.
- Do not substitute mAP or classification accuracy for counting accuracy.
- Never tune hyperparameters or thresholds on the final test set.
- Validate full-image inference, not just isolated tiles.
- Verify real Flask integration with test results, not screenshots alone.
- Report hard examples, errors, uncertainty and known limitations.
- Provide reproducible settings, dataset provenance, weights and execution instructions.

12. Developer completion checklist

☐ Notebook runs from start to finish

☐ Data provenance and annotation quality documented

☐ Train/validation/test source splits verified

☐ YOLO weights and settings recorded

☐ Detection metrics calculated

☐ Counting metrics calculated

☐ Threshold chosen on validation only

☐ 640x640 tiling and boundaries verified

☐ Deduplication behaviour tested

☐ Robustness and unseen imagery evaluated

☐ Baseline and improvements compared

☐ Flask integration tested

☐ Latency and memory measured

☐ Charts interpreted and limitations documented

☐ Final readiness decision justified

Final instruction to developer

Deliver a research-quality, reproducible Jupyter Notebook that establishes detection accuracy, tree-count accuracy, generalisation to unseen drone imagery and end-to-end performance in the existing AI First Nations Flask application. Trustworthy evaluation and transparent limitations matter more than an artificially high accuracy number.